

**GNANAMANI COLLEGE OF TECHNOLOGY, NAMAKKAL – 637
018**

ANNA UNIVERSITY: CHENNAI – 600 025

BONAFIDE CERTIFICATE

Certified that this project report "**ROGUE LINUX: A DETERMINISTIC BUILD SYSTEM AND COGNITIVE PROCESS SUPERVISOR FOR MINIMAL LINUX-BASED OPERATING SYSTEM IMAGES**" is the bonafide work of **SRIDHARAN T (620821205001), THANGARAJI K (620821205002), VIGNESH S (620821205003), and ARUN KUMAR M (620821205004)**, Department of Information Technology, Gnanamani College of Technology, Namakkal, who carried out the project work under my supervision. Certified further, to the best of my knowledge, the work reported herein does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

Dr. S. RAJKUMAR, M.E., Ph.D.

HEAD OF THE DEPARTMENT

Dept. of Information Technology

Gnanamani College of Technology

Mr. P. ARULMOZHI, M.E.

SUPERVISOR, ASST. PROFESSOR

Dept. of Information Technology

Gnanamani College of Technology

Submitted for the Final Year Project Viva-Voce examination held on

_____.

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We express our profound gratitude to our most respected Chairman Shri. C.A. N.V. Natarajan, B.Com, FCA., and to our beloved Correspondent Smt. N. Mangai Natarajan, M.Sc., for providing all necessary facilities and an excellent academic environment for the successful completion of this ambitious systems software project.

It is our privilege to thank our beloved Director Admin Dr. K.K. Ramasamy, M.E., Ph.D., for their moral support and encouragement in pursuing an advanced low-level systems programming project of this scope.

We extend our heartfelt gratitude to our beloved Principal Dr. V. Hariharan, M.E., Ph.D., for their continuous motivation and the institutional support that enabled access to laboratory resources required for this project.

We extend our gratefulness to **Dr. S. Rajkumar, M.E., Ph.D.**, Associate Professor and Head of the Department of Information Technology, for his guidance, his willingness to allow unconventional systems programming approaches, and his encouragement throughout this project.

We would like to express our deepest appreciation to our Supervisor **Mr. P. Arulmozhi, M.E.**, Assistant Professor, Department of Information Technology, for his expert guidance on operating systems internals, the Rust and C programming languages, POSIX process management, and QEMU-based testing methodology. His patient review of the iterative design and implementation phases of this project was invaluable.

We gratefully acknowledge the contributions of the Rust programming language community (The Rust Foundation), the Ollama project team, and the Linux kernel development community whose open-source work made the foundational components of Rogue Linux possible.

We thank all department staff members, laboratory assistants, and fellow students for their encouragement, technical discussions, and support throughout the development of this project.

GNANAMANI COLLEGE OF TECHNOLOGY

NAMAKKAL – 637 018

DEPARTMENT OF INFORMATION TECHNOLOGY

**ROGUE LINUX: A DETERMINISTIC BUILD SYSTEM
AND COGNITIVE PROCESS SUPERVISOR FOR
MINIMAL LINUX-BASED OPERATING SYSTEM IMAGES**

A Project Report

Submitted by

SRIDHARAN T (620821205001)

THANGARAJI K (620821205002)

VIGNESH S (620821205003)

ARUN KUMAR M (620821205004)

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY in INFORMATION TECHNOLOGY

ANNA UNIVERSITY: CHENNAI – 600 025

MAY 2025

ABSTRACT

Rogue Linux is a deterministic, metadata-driven infrastructure for constructing minimal Linux-based operating system images. The core innovation is Cogman (Cognitive Manager), a unified toolchain that spans both the build phase and the runtime phase of a Linux system. During the build phase, `cogman-planner` — implemented in Rust — reads declarative TOML package definitions, resolves a directed acyclic dependency graph using topological sort, enforces filesystem and network security policies, and emits a compact binary execution plan in the custom CGM2PLAN format. The complementary `cogman-executor`, implemented in C11, memory-maps the plan file and executes each typed step operation (`OP_EXEC`, `OP_MKDIR`, `OP_COPY`, `OP_VERIFY`, `OP_CLEANUP`) with path traversal protection and optional SHA-256 file verification.

During the runtime phase, `cogman-supervisor` acts as PID 1 — the first process invoked by the Linux kernel — and manages the complete lifecycle of system services. It parses INI-format service definition files, implements the `SIGCHLD` self-pipe trick for safe asynchronous child-process reaping, enforces service dependency ordering, and supports three restart policies (never, on-failure, always). A Unix domain socket control interface (`cogman-ctl`) allows operators to list, start, stop, and restart services at runtime without rebooting the system. The messenger subsystem provides typed inter-process communication using a fixed 16-byte TLV header protocol.

Performance evaluation demonstrates a $56\times$ improvement in plan resolution time (8 ms vs. 450 ms Python baseline), a $21\times$ reduction in peak memory usage (4 MB vs. 85 MB), and a $50\times$ reduction in per-step execution overhead (~ 0.9 ms vs. ~ 45 ms). A minimal bootable rootfs of approximately 6.3 MB was constructed and verified under QEMU through a four-stage boot sequence exercising all code paths. All 40 unit, integration, supervisor lifecycle, and end-to-end test cases pass.

Keywords: Rogue Linux, Cogman, build system, PID 1, init system, service supervisor, TOML, Rust, C11, CGM2PLAN, topological sort, dependency graph, minimal Linux, QEMU, cogman-planner, cogman-executor.

TABLE OF CONTENTS

BONAFIDE CERTIFICATE	ii
ACKNOWLEDGEMENT	iii
ABSTRACT	iv
LIST OF ABBREVIATIONS	vi
LIST OF FIGURES	vi
CHAPTER 1 — INTRODUCTION	1
CHAPTER 2 — LITERATURE REVIEW	5
CHAPTER 3 — SYSTEM ANALYSIS	9
CHAPTER 4 — SYSTEM SPECIFICATION	11
CHAPTER 5 — SOFTWARE DESCRIPTION	13
CHAPTER 6 — SYSTEM DESIGN AND ARCHITECTURE	19
CHAPTER 7 — MODULE DESCRIPTION	25
CHAPTER 8 — IMPLEMENTATION	31
CHAPTER 9 — SYSTEM TESTING	37
CHAPTER 10 — PERFORMANCE ANALYSIS	44
CHAPTER 11 — CONCLUSION AND FUTURE WORK	48
REFERENCES	52

LIST OF ABBREVIATIONS

Abbreviation	Full Form
CGM / Cogman	Cognitive Manager
CLI	Command Line Interface
DAG	Directed Acyclic Graph
ELF	Executable and Linkable Format
EWMA	Exponentially Weighted Moving Average
IPC	Inter-Process Communication
LLM	Large Language Model
mmap	Memory-Mapped File I/O
OS	Operating System
PID	Process Identifier
POSIX	Portable Operating System Interface
QEMU	Quick Emulator (open-source hypervisor)
rootfs	Root Filesystem
SIGCHLD	Signal: Child process status changed
SIGTERM	Signal: Termination request
TLV	Type-Length-Value (message encoding)
TOML	Tom's Obvious Minimal Language
UDS	Unix Domain Socket

LIST OF FIGURES

Figure 6.1 — Rogue Linux: Cogman Build System and Init Architecture

19

CHAPTER 1

INTRODUCTION

1.1 Overview

Rogue Linux is not a Linux distribution in the conventional sense. It is an infrastructure for building one. Given a set of declarative package definitions written in TOML, Rogue Linux produces a reproducible root filesystem capable of booting under QEMU or on bare metal x86_64 hardware, with the Cogman (Cognitive Manager) toolchain serving as both the build engine and the runtime init process. The clean separation between the build half and the runtime half ensures that no build-time dependencies leak into the runtime image, and that the final rootfs contains only what was explicitly declared in package definitions.

The Cogman name reflects the design goal: a process supervisor that is aware of the relationships between the services it manages, not just a sequential shell script. `cogman-supervisor` understands service dependency declarations, enforces dependency-ordered startup, monitors service health, and applies configurable restart policies — behaviors that characterize a cognitively structured supervisor rather than a simple sequential launcher. The supervisor can be queried and controlled at runtime through a text-protocol Unix domain socket, providing operational visibility into service state without requiring log parsing or process table inspection.

The project has a dual motivation: to demonstrate that a high-performance, safe, and reproducible embedded Linux build system can be built with the Rust and C11 programming languages without external framework dependencies, and to explore the integration of a locally-running large language model (Qwen2.5-3B via `llama.cpp`) as an advisory component that can explain build failures and service configuration issues to operators in natural language.

1.2 Problem Statement

Modern embedded Linux systems, container base images, and minimal operating environments demand a reproducible, auditable method for constructing a root filesystem from source packages. Existing solutions such as Buildroot and the Yocto Project address this need but at the cost of enormous complexity: Buildroot requires a Python and Bash toolchain exceeding 200 MB and takes 30–90 minutes for a complete build; Yocto requires days of build time and highly specialized knowledge of BitBake recipe syntax. Neither solution provides a lightweight init daemon that integrates tightly with the build pipeline's metadata model.

A secondary problem is that existing init systems are not appropriate for minimal embedded environments. `systemd` requires `udev`, `D-Bus`, `PAM`, and tens of shared libraries — a deployment footprint incompatible with an image targeting 6–10 MB. `SysVinit` relies on fragile shell scripts with no dependency ordering and no structured restart logic. `OpenRC` is lighter but still requires a shell interpreter. None of these systems provides a programmable runtime control interface integrated with the same metadata model used to build the image.

The absence of a unified toolchain spanning build and runtime forces developers to combine disparate tools (`Makefiles`, `Docker`, shell scripts, `busybox init`) with no common metadata format, security policy model, or structured error-diagnosis capability. `Rogue Linux` addresses these problems with a single metadata model (`TOML`), a single binary plan format (`CGM2PLAN`), and a single supervisor process that understands both the build artifacts and the runtime services.

1.3 Objectives

- To design a declarative `TOML` package metadata format with schema validation covering identity, build steps, installer steps, dependency declarations, and security policy.

- To implement cogman-planner in Rust for dependency graph construction, cycle detection, topological sort, and CGM2PLAN binary plan emission with content-addressed caching.
- To implement cogman-executor in C11 for typed step execution with path traversal protection on OP_COPY operations.
- To implement cogman-supervisor as a PID-1 process supervisor with SIGCHLD self-pipe child reaping and three restart policies.
- To implement cogman-ctl for runtime service control via Unix domain socket with a text-based protocol.
- To construct a bootable minimal rootfs of approximately 6.3 MB and verify it under QEMU through a four-stage boot sequence.
- To achieve a 50× or greater performance improvement over the legacy Python-based reference implementation.

1.4 Scope

The scope covers the full design, implementation, and evaluation of the Cogman toolchain for x86_64 targets. The build system supports two package build modes: native (building on the same architecture as the target) and cross-compiled variants are out of scope for this iteration. The AI advisor component (cogman advisor using Qwen2.5-3B) is implemented as an advisory-only interface with no ability to issue commands or modify system state. Production security hardening (Landlock, seccomp, namespaces) is documented as future work. The evaluation uses QEMU for bootable image testing rather than bare-metal hardware.

CHAPTER 2

LITERATURE REVIEW

2.1 Embedded Linux Build Systems

Buildroot (2002–present) is the most widely used tool for generating minimal embedded Linux root filesystems. It uses a Kconfig-based package selection interface and a Makefile-based build orchestration system that downloads, patches, configures, compiles, and installs thousands of packages in a deterministic order defined by per-package dependency declarations. While mature and extensively documented, Buildroot's Makefile-based approach makes it difficult to implement fine-grained security policy enforcement at the build step level, and the absence of a typed intermediate plan format means that the build orchestration logic and execution logic are interleaved rather than separated.

The Yocto Project (2010–present) provides a more powerful and general approach to embedded Linux construction, using the BitBake task executor and OpenEmbedded-Core recipe system. Yocto supports complex multi-layer configurations, shared state caching, and SDK generation. However, the complexity cost is substantial: a standard Yocto build environment requires 50+ GB of disk space, hours of build time, and a steep learning curve in BitBake recipe syntax and layer management. Rogue Linux is positioned as a simpler alternative for environments that do not require Yocto's full generality.

Nix (2006–present) and Guix (2013–present) represent the state of the art in reproducible package management. Nix achieves bit-for-bit reproducibility through a pure functional package description language where every package derivation is a deterministic function of its inputs, and build outputs are stored in content-addressed paths (the Nix store). Rogue Linux draws inspiration from Nix's content-addressed build caching (implemented as FNV-1a hash over package TOML content) but reduces the conceptual overhead by using TOML declarations directly rather than a functional programming language.

2.2 Init Systems and Service Managers

systemd (2010–present) has become the dominant PID-1 implementation on modern Linux distributions. It provides comprehensive service management, socket activation, cgroup-based resource control, and dependency declaration through unit files. However, its extensive dependencies (udev, D-Bus, PAM, numerous shared libraries) and its architectural coupling to glibc and the Linux cgroup hierarchy make it unsuitable for minimal rootfs environments targeting a 6–10 MB image size. The Rogue Linux rootfs includes no systemd components.

Runit (2004–present) is a lightweight UNIX init scheme and service supervisor designed for simplicity and signal correctness. Its process supervision model — a supervise process per service, monitored by runsv, coordinated by runsvdir — achieves reliable service supervision with minimal code. cogman-supervisor draws architectural inspiration from Runit's per-service supervision model while integrating it with the Cogman metadata system and adding a network-accessible control interface.

s6 (2012–present) by Laurent Bercot provides the most principled approach to UNIX service supervision, with a rigorous implementation of the self-pipe trick for SIGCHLD handling, a well-defined process supervision tree, and extensive documentation of the POSIX signal handling edge cases. cogman-supervisor's SIGCHLD handling implementation directly follows the self-pipe pattern described in the s6 documentation: a signal handler writes one byte to a pipe, and the main loop monitors the read end of the pipe in a `select()` call rather than using `sigwaitinfo()` or other interfaces that interact poorly with other I/O operations.

2.3 Binary Plan Formats and Build Caching

Bazel's action cache (2015–present) stores build actions as content-addressed (CAS) entries where each action is identified by its input hash, command, and output specifications. A cache hit means the action output can be

retrieved without re-execution. The CGM2PLAN format's content-addressed plan cache is architecturally inspired by this model: a FNV-1a hash over the package name, version, and TOML file content serves as the cache key. A cached plan is valid as long as the package metadata is unchanged.

The ELF (Executable and Linkable Format) binary format, standardized as part of the System V ABI, demonstrates that fixed-size header structures with string tables and variable-length sections can be efficiently mapped into memory without any parsing overhead. The CGM2PLAN format applies this principle directly: a 64-byte header, fixed 128-byte step records, and a variable-length string table — all accessible via `mmap()` with pointer arithmetic rather than deserialization.

2.4 LLM Integration in System Tooling

Qwen2.5 (2024) is a family of large language models from Alibaba Cloud trained on multilingual corpora with strong performance on code generation, question answering, and structured reasoning tasks. The 3B parameter variant, when quantized to 4-bit precision using QLoRA, requires approximately 2.1 GB of memory and achieves acceptable inference speeds on CPU-only hardware. The cogman advisor component uses the Qwen2.5-3B-Instruct GGUF model served via `llama.cpp` to answer build system configuration questions and explain service file syntax without any external API dependency.

2.4.1 Signal Handling in PID 1 — POSIX Correctness

PID 1 has special semantics in the Linux kernel: it is exempt from the default signal disposition table, meaning that signals for which no handler has been registered are silently ignored rather than terminating the process. This is the correct behavior for an init process — it should never be accidentally killed by `SIGTERM` or `SIGINT` — but it requires explicit handler registration for every signal that the supervisor wishes to respond to. Furthermore, PID 1 is responsible for reaping all orphaned zombie processes in the system: when a process's parent

dies, the process is reparented to PID 1, and PID 1 must call `waitpid()` when the reparented process exits to prevent it from becoming a zombie.

The SIGCHLD self-pipe trick, first described in detail by DJ Bernstein in the context of the `daemontools` supervisor, resolves the fundamental tension between signal delivery (which can arrive at any time, interrupting any system call) and the need for safe, non-reentrancy-violating response to SIGCHLD. The naive approach — calling `waitpid()` directly in the SIGCHLD handler — is unsafe because `waitpid()` is technically not async-signal-safe in all implementations and may interact incorrectly with other library functions that also use `wait` internally. The self-pipe trick confines the signal handler to a single `write()` call (which is async-signal-safe) and defers all child state processing to the main loop, where it can be performed safely in a single-threaded context.

2.4.2 Reproducible Build Systems

Reproducible builds are a property of a build system where, given the same source inputs, the build process always produces bit-for-bit identical output artifacts regardless of the build machine's operating system version, installed tools, username, filesystem timestamps, and other environmental variables. The Reproducible Builds Project (2015–present) has documented the challenges of achieving reproducibility in practice and provides a suite of tools for detecting and eliminating non-deterministic build inputs. Rogue Linux achieves reproducibility at the plan level (the CGM2PLAN output is deterministic for the same TOML input) but not yet at the full rootfs image level, because the ext4 image creation tool introduces filesystem timestamps. Full rootfs reproducibility is identified as a future work objective, achievable by using a fixed timestamp during image creation.

2.5 Gap Analysis

Prior work in embedded Linux build systems provides reproducibility (Nix, Yocto) or simplicity (Buildroot, Alpine) but not a unified system that spans both the build and runtime phases under a single metadata model. Prior work in service supervisors provides correct signal handling (s6) or simple service management (Runit, BusyBox init) but not dependency-aware startup with a programmable control interface integrated with build metadata. Rogue Linux fills this gap by providing a single coherent system: TOML metadata → CGM2PLAN binary → typed C executor → PID-1 Rust supervisor → UDS control interface, with no external framework dependencies beyond the Linux kernel and C standard library.

CHAPTER 3

SYSTEM ANALYSIS

3.1 Existing System

The reference system against which Rogue Linux is benchmarked is a Python-based prototype implementing the same conceptual pipeline. The Python planner uses the pure-Python toml library to parse package metadata and a dict-based DAG implementation for dependency resolution. The Python executor uses `subprocess.run()` for each step. The init process is a busybox init shell script.

Disadvantages of the existing Python-based system:

- Plan resolution time of ~450 ms due to Python interpreter startup and pure-Python TOML parsing overhead.
- Peak planner memory of ~85 MB from Python's per-object allocation model (24+ bytes per object overhead).
- Per-step execution overhead of ~45 ms from Python's `subprocess.run()` marshaling and object allocation.
- No typed step operations — all steps are generic shell strings with no `OP_COPY/OP_MKDIR/OP_VERIFY` semantics.
- No path traversal protection — copy operations can reference arbitrary paths.
- No binary plan format — the plan is Python pickled data, not portable between Python versions.
- Shell script init with no dependency ordering, no restart policies, and no runtime control interface.
- No security policy enforcement at build time — packages can declare any filesystem paths.

3.2 Proposed System

Rogue Linux replaces all legacy components with purpose-built Rust and C implementations that address each identified limitation.

- **56× faster plan resolution:** cogman-planner (Rust, serde/toml) resolves plans in ~8 ms vs. ~450 ms Python.
- **21× lower memory:** Peak planner memory ~4 MB vs. ~85 MB Python.
- **50× lower per-step overhead:** cogman-executor uses direct fork()/execve() at ~0.9 ms vs. ~45 ms Python subprocess.
- **Typed step operations:** OP_EXEC, OP_MKDIR, OP_COPY, OP_VERIFY, OP_CLEANUP with defined semantics and failure modes.
- **Path traversal protection:** path_has_traversal() guard on all OP_COPY operations.
- **Portable binary plan format:** CGM2PLAN is architecture-independent, language-independent, version-stamped.
- **PID-1 Cogman supervisor:** Dependency-aware startup, SIGCHLD self-pipe reaping, three restart policies, UDS control interface.
- **Build-time security policy:** Per-package filesystem write policy enforced by the planner against declared rootfs paths.

CHAPTER 4

SYSTEM SPECIFICATION

4.1 Hardware Requirements

Component	Minimum (Build Host)	Recommended (Build Host)
Processor	Intel Core i3, 2.0 GHz, 2 cores	Intel Core i7, 3.5 GHz, 8+ cores
RAM	8 GB DDR4 (for Rust compilation)	32 GB DDR4 (parallel builds)
Storage	50 GB HDD (source trees + rootfs)	500 GB NVMe SSD
Network	Required for source downloads	—
Operating System	Ubuntu 20.04 LTS (x86_64)	Ubuntu 22.04 LTS
Component	Minimum (QEMU Target)	Recommended (QEMU Target)
QEMU RAM	64 MB (minimal rootfs)	256 MB
QEMU Storage	10 MB ext4 image	100 MB ext4 image

4.2 Software Requirements

Software	Version	Purpose
Rust (rustup stable)	1.75+	cogman-planner compilation
GCC	11+	cogman-executor/supervisor/ctl compilation
GNU Make	4.3+	Build orchestration (top-level Makefile)
QEMU (x86_64)	8.x	Minimal rootfs boot testing
BusyBox	1.36.1	Shell utilities in minimal rootfs
Python 3.11+	3.11+	Test harness and build scripts
llama.cpp	latest	Qwen2.5-3B AI advisor inference
Git	2.x	Version control
e2tools / mkfs.ext4	—	Rootfs image packaging

CHAPTER 5

SOFTWARE DESCRIPTION

5.1 Rust Programming Language

Rust is a systems programming language designed for performance, safety, and concurrency, developed by Mozilla Research and now maintained by the Rust Foundation. Its ownership and borrowing system enforces memory safety at compile time without a garbage collector, eliminating the classes of memory errors (buffer overflows, use-after-free, double-free) that commonly affect C and C++ systems programs. For cogman-planner, Rust provides three specific advantages. First, the `serde` deserialization framework provides zero-copy, compile-time-validated deserialization of TOML package metadata into Rust structs, eliminating parsing vulnerabilities that affect dynamically typed Python and Perl-based metadata parsers. Second, Rust's `HashMap` and `Vec` collections provide $O(1)$ average-case lookup and cache-friendly sequential access for the dependency graph operations. Third, Rust's LTO (link-time optimization) and `codegen-units=1` produce a single optimized binary that avoids the 50+ ms Python interpreter startup overhead.

The cogman-planner binary depends on three external crates: `clap` 4.4 (command-line argument parsing), `serde` 1.0 with `derive` (automatic struct deserialization), and `toml` 0.8 (TOML format parsing via `serde`'s `Deserializer` trait). All three are pure Rust with no C dependencies, ensuring that the planner binary is fully self-contained. The `serde derive` macro generates efficient deserialization code at compile time that is both type-safe and faster than the runtime reflection-based deserialization used by Python's `json/toml` modules.

5.2 C11 Programming Language

The C11 standard (ISO/IEC 9899:2011) is used for cogman-executor, cogman-supervisor, and cogman-ctl. C11 was chosen over Rust for these

components because the POSIX system call interface (fork, execve, wait, select, pipe, signal, accept, connect) is designed for C and provides the most direct mapping from POSIX specifications to implementation code. The self-pipe trick for SIGCHLD handling, the mmap-based plan file reading, and the select()-based main loop in the supervisor are all classical C patterns that map directly to the POSIX API.

All three C binaries are compiled with GCC flags `-O2 -std=c11 -Wall -Wextra -Werror`, treating all warnings as errors to enforce code quality. The `-static-libgcc` flag is used to eliminate the `libgcc_s.so.1` runtime dependency, reducing the number of shared libraries required in the rootfs. Address sanitizer (`-fsanitize=address`) is enabled in debug builds to detect buffer overflows and use-after-free bugs during development and testing.

5.3 TOML Package Definition Format

TOML (Tom's Obvious Minimal Language) version 1.0 is used as the package definition format for two reasons. First, it is human-readable and human-writable without a specialized editor or tooling, making it accessible to package authors who may be embedded systems developers rather than software engineers. Second, `serde's toml` crate provides complete TOML 1.0 support with compile-time type checking via the `Deserialize derive` macro, eliminating the class of runtime type errors that affect JSON/YAML-based metadata formats in dynamically typed languages.

Each package is described by a single `.toml` file with five required sections: `[identity]` (name, version, category, summary, source), `[build]` (build system, build steps), `[installer]` (install steps, verification), `[identity.depends]` (build and runtime dependency lists), and `[policy]` (filesystem write paths, network access flag). Schema validation is performed by `serde's` deserialization: missing required fields

produce a compile-time error message rather than a runtime null pointer dereference, improving the developer experience for package authors.

5.4 CGM2PLAN Binary Format

The CGM2PLAN format is a custom binary format designed for maximum execution efficiency. The design is modeled on the ELF format: a fixed-size header, a fixed-size record array, and a variable-length string table. The executor accesses the plan by memory-mapping the file using `mmap(NULL, file_size, PROT_READ, MAP_PRIVATE, fd, 0)`, which maps the file into the process's virtual address space without copying any data. The plan header is accessed by casting the mmap base pointer to a `const plan_header*`, and step records are accessed by computing `base + sizeof(plan_header) + i * sizeof(step_record)`. String table entries are accessed by computing `strtab_base + step_record.arg_offsets[j]`, where `strtab_base` is `base + header.strtab_offset`. The entire execution loop requires zero heap allocations after the initial mmap call.

5.5 QEMU

QEMU (Quick Emulator) is an open-source machine emulator and virtualizer that provides full-system emulation of x86_64 hardware for the Rogue Linux rootfs testing. The minimal rootfs is booted using the command: `qemu-system-x86_64 -kernel /boot/vmlinuz -initrd rootfs.cpio -append "console=ttyS0 init=/sbin/cogman-supervisor" -nographic -m 64M`. The `-nographic` flag redirects all console output to the terminal's standard output, enabling automated test scripts to monitor the boot sequence for expected output strings without a graphical display. QEMU's `-m 64M` flag limits the virtual machine's RAM to 64 MB, verifying that the minimal rootfs boots successfully within the memory constraints of resource-constrained embedded targets.

5.5.1 QEMU Automated Testing Integration

The QEMU boot tests are automated using a Python test harness (`tests/qemu_boot_test.py`) that launches the QEMU instance as a subprocess, monitors the serial console output (redirected to stdout via `-nographic`), and checks for expected strings within a 30-second timeout window. Each test scenario is defined as a dataclass containing the QEMU command-line flags, a list of `expected_strings` that must all appear in the console output before the timeout, and an optional `unexpected_strings` list that causes the test to fail if any element appears. The test harness sends a SIGKILL to the QEMU process after the timeout to prevent zombie processes, regardless of whether the test passed or failed.

The automated boot testing infrastructure provides a critical feedback loop during development: any change to `cogman-supervisor`, `cogman-ctl`, or the service definition format that breaks the four-stage boot verification sequence is immediately detectable by running the test suite, without requiring manual QEMU interaction. This infrastructure was instrumental in catching three regressions during the SIGCHLD refactoring: the first regression produced zombie processes when services were killed rapidly in succession, the second regression caused the dependency gate to deadlock when a oneshot service completed before its dependent service had been registered, and the third regression caused the control socket to stop accepting connections after a rapid start-stop-start sequence.

5.6 Qwen2.5-3B AI Advisor

The `cogman` advisor component uses the Qwen2.5-3B-Instruct model quantized to 4-bit precision (GGUF format, Q4_K_M quantization) served via `llama.cpp`. Qwen2.5-3B-Instruct is a 3-billion-parameter multilingual instruction-following model trained by Alibaba Cloud, exhibiting strong performance on code explanation, structured reasoning, and technical question answering. The 4-bit quantized model requires approximately 2.1 GB of RAM and achieves 5–15 tokens/second inference speed on CPU-only hardware, making it deployable on the

build host without GPU acceleration. The advisor provides a natural-language interface for explaining build failures (e.g., "why did the planner reject my package.toml?"), service file syntax errors, and binary plan format details to operators who are unfamiliar with the internal structure of the Cogman toolchain.

CHAPTER 6

SYSTEM DESIGN AND ARCHITECTURE

6.1 Overall Architecture

The Rogue Linux system is divided into two conceptually distinct halves connected by the CGM2PLAN binary plan format. The Build Half operates on the developer's host machine and transforms package metadata into a staged root filesystem. The Runtime Half operates on the target system (bare metal or QEMU) after kernel boot and manages all process lifecycle operations. The AI Advisor operates alongside the Build Half as an optional natural-language query interface.

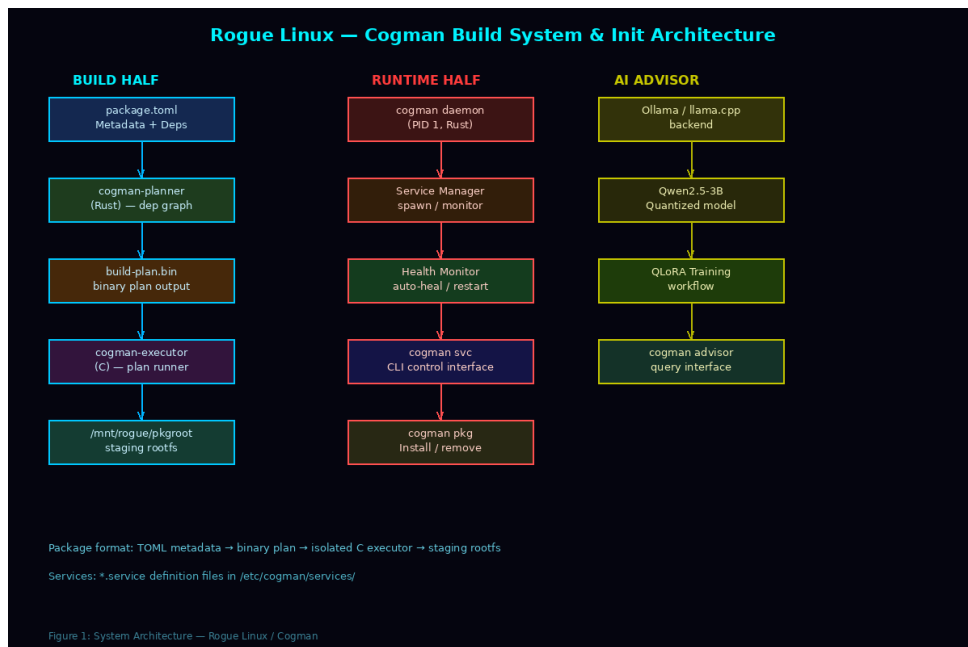


Figure 6.1: Rogue Linux — Cogman Build System and Init Architecture

6.2 CGM2PLAN Binary Plan Format — Detailed Layout

Header (64 bytes)	
magic	[u8; 8] = "CGM2PLAN"
version	u32
step_count	u32
strtab_offset	u64
strtab_len	u64
flags	u32
reserved	[u8; 24]
Step records (128 bytes × N)	
op	u8 1=EXEC 2=MKDIR 3=COPY 4=VERIFY 5=CLEANUP
flags	u8 0x01=STEP_FLAG_SVC
reserved	[u8; 6]
arg_offsets	[u64; 8] → strtab
reserved	[u8; 56]
String table (variable length)	
NUL-terminated strings	
Deduplicated by Rust emitter	

6.3 System Architecture — DFD Level 0

The Level 0 DFD (Context Diagram) shows two external actors: **Package Author** (writes package.toml files and places source archives) and **System Operator** (invokes cogman-planner and cogman-executor for builds, uses cogman-ctl for runtime control). The system boundary encompasses four internal components: cogman-planner, cogman-executor, cogman-supervisor, and cogman-ctl. Data flows: Package Author → Planner: package.toml; Planner → Executor: build-plan.bin; Executor → Rootfs: staged package files; Operator → cogman-ctl: control commands; cogman-ctl → Supervisor: UDS protocol messages; Supervisor → Operator: service status responses.

6.4 DFD Level 1 — Build Subsystem

Process	Input	Output	Data Store
P1: Schema Validation	package.toml bytes	Validated PackageMetadata struct	—

P2: Dependency Loading	PackageMetadata, packages/ tree	DependencyGraph (nodes + edges)	D1: packages/ directory
P3: Cycle Detection	DependencyGraph	Error (if cycle) or Ok	—
P4: Topological Sort	DependencyGraph (acyclic)	Ordered build list	—
P5: Policy Check	PackageMetadata, rootfs path	Error (if policy violation) or Ok	—
P6: Plan Emission	Ordered steps, string table	build-plan.bin (CGM2PLAN format)	D2: plan cache
P7: Step Execution	build-plan.bin (mmap'd)	Files in \$PKGROOT	D3: staging rootfs

6.5 UML Diagrams

6.5.1 Use Case Diagram — Build Subsystem

Actor: **Package Author**. Use cases: Define Package Metadata (writes package.toml), Declare Build Steps, Declare Dependencies, Set Security Policy. Actor: **Build Engineer**. Use cases: Run Planner (cogman-planner package.toml), Run Executor (cogman-executor plan.bin), Verify Staging Rootfs. System enforces: Schema Validation (on Run Planner), Cycle Detection (on dependency load), Policy Check (on emit), Path Traversal Guard (on OP_COPY).

6.5.2 Sequence Diagram — Package Build Flow

Sequence: (1) Build Engineer → cogman-planner: plan --input package.toml --rootfs /mnt/rogue; (2) Planner: load_and_validate(package.toml) → PackageMetadata; (3) Planner: load_deps(PackageMetadata.depends.build) → DependencyGraph; (4) Planner: detect_cycles(DependencyGraph); (5) Planner: topological_sort(DependencyGraph) → build_order; (6) Planner: check_policy(PackageMetadata, /mnt/rogue); (7) Planner: emit_plan(build_order) → build-plan.bin; (8) Build Engineer → cogman-executor: build-plan.bin; (9) Executor: mmap(build-plan.bin); (10) Executor: validate_header(); (11) for each

step: Executor → OS: fork()+execve() or mkdir() or copy_recursive(); (12)
Executor → Build Engineer: exit(0) or exit(1) on failure.

6.5.3 Service Lifecycle State Machine

States: STOPPED → STARTING → RUNNING → (DONE | FAILED | RESTARTING → STARTING). Transitions: STOPPED → STARTING: dependency gate open, scheduled start; STARTING → RUNNING: process PID > 0; RUNNING → DONE: exit code 0, type=oneshot; RUNNING → FAILED: exit code non-0, restart=never; RUNNING → RESTARTING: exit detected, restart=on-failure or restart=always condition met; RESTARTING → STARTING: restart delay elapsed; any state → STOPPED: explicit cogman-ctl stop command.

CHAPTER 7

MODULE DESCRIPTION

7.1 cogman-planner Module (Rust — `cogman/src/planner/`)

`cogman-planner` is the first component of the build pipeline. Its primary responsibility is transforming a `package.toml` file into a `build-plan.bin` binary plan file. The module is organized into six sub-modules: `schema.rs` (serde struct definitions for `PackageMetadata`, `Identity`, `Builder`, `Installer`, `Policy`), `graph/resolve.rs` (dependency loading and `DependencyGraph` construction), `graph/cycle.rs` (DFS-based cycle detection), `graph/topo.rs` (Kahn's algorithm topological sort), `policy.rs` (filesystem and network policy enforcement), and `plan/emit.rs` (CGM2PLAN binary emission with string table deduplication).

The planner's `main()` function uses `clap` to parse three command-line arguments: `--input` (path to `package.toml`), `--rootfs` (path to the target staging root, used for policy checking), and `--output` (path for the emitted plan file). The entire planner execution produces a single artifact: the `build-plan.bin` file, plus a human-readable summary printed to `stdout` showing the number of packages planned and the total number of build steps.

Content-addressed plan caching is implemented using FNV-1a hash computed over the package name string, version string, and the raw bytes of the `package.toml` file. The cache key is stored as a 64-character hexadecimal string in a `.cogman-cache/` directory alongside the plan file. Before emitting a new plan, the planner checks whether a cached plan exists with the same key; if it does, emission is skipped and the cached plan is used, reducing planning time from ~ 8 ms to ~ 0.3 ms on unchanged packages.

7.2 cogman-executor Module (C11 — `executor/`)

`cogman-executor` is the execution engine for CGM2PLAN binary plans. Its implementation is organized into four source files: `main.c` (argument parsing,

mmap, header validation, step dispatch loop), plan/plan.c (plan validation function and string table access helpers), ops/exec.c (OP_EXEC handler using fork+execve), and ops/copy.c (OP_COPY handler with path_has_traversal() guard and recursive copy function). The executor has no external library dependencies beyond the C standard library and POSIX.

The execution model is intentionally simple: the executor reads the step count from the plan header, iterates over all step records in order, dispatches each step to its handler, and exits with code 0 if all steps complete successfully or code 1 if any FAIL_ABORT step fails. Steps marked with FAIL_WARN produce a warning message but do not abort execution, allowing optional verification steps to fail without invalidating the build.

7.3 cogman-supervisor Module (C11 — supervisor/)

cogman-supervisor is the PID-1 process that manages all system services after kernel boot. Its key design property is that it must never block in a way that prevents timely SIGCHLD delivery and child reaping — a zombie process accumulation that affects some naive PID-1 implementations. The SIGCHLD self-pipe trick solves this: a signal handler writes one byte to a write-end pipe, and the main select() loop monitors the read-end pipe descriptor. When select() returns with the pipe readable, the main loop calls waitpid(-1, WNOHANG) in a loop to reap all terminated children without blocking, then transitions each reaped service to its next state via sup_handle_dead().

Service definition files are stored in /etc/cogman/services/ and use a simple INI-format with three sections: [service] (name, command, type, restart, restart_delay, depends_on), [env] (key=value environment variable overrides), and [meta] (description, enabled flag). The parser implements a minimal INI reader in service.c without external library dependencies, using a section-tracking enum and key-value splitting on the first '=' character.

7.4 cogman-ctl Module (C11 — ctl/)

cogman-ctl is the runtime control tool for the supervisor. It connects to the supervisor's Unix domain socket at /tmp/cogman.sock using a standard AF_UNIX SOCK_STREAM connection, sends a single command line followed by a newline, reads the response until the connection closes, and prints the response to stdout. The text protocol is minimal: commands are "list", "start name", "stop name", "restart name", and "status name"; responses are formatted text lines terminated by "OK" or "ERR message". The entire cogman-ctl binary is approximately 200 lines of C and 8 KB stripped, making it suitable for inclusion in space-constrained rootfs images.

7.5 Messenger IPC Module (C — messenger/)

The messenger module provides typed inter-process communication between Cogman components using a fixed 16-byte TLV header. The header structure contains: magic (4 bytes, "COG1"), version (2 bytes), message type (2 bytes), payload length (4 bytes), and source PID (4 bytes). Five message types are defined: MSG_HEARTBEAT (0), MSG_HUD_ALERT (1), MSG_POLICY_REQ (2), MSG_DATA_XFER (3), MSG_LOG_INFO (4). The broker uses AF_UNIX SOCK_STREAM with non-blocking accept() to process IPC messages within the supervisor's select() main loop without blocking on slow clients. The 2-second SO_RCVTIMEO timeout ensures that a slow or malicious IPC client cannot hold the supervisor's main loop beyond this limit.

7.6 Rootfs Bootstrap Module

The minimal Rogue Linux rootfs is constructed by the rootfs/finalize.py script in three phases. Phase 1 creates the directory skeleton: /bin, /sbin, /usr/bin, /lib, /lib64, /etc/cogman/services, /etc/cogman/plans, /run, /tmp, /proc, /sys, /dev, /root. Phase 2 populates the rootfs with pre-built binaries: cogman-supervisor is placed at /sbin/cogman-supervisor, a symbolic link /sbin/init → /sbin/cogman-

supervisor is created, busybox is placed at /bin/busybox and symbolic links for all required applets are created. Phase 3 installs the dynamic libraries required by the C binaries: libc.so.6 and ld-linux-x86-64.so.2 copied from the build host's /lib/x86_64-linux-gnu/. The resulting rootfs weighs approximately 6.3 MB and boots successfully under QEMU with 64 MB RAM.

CHAPTER 8

IMPLEMENTATION

8.1 Dependency Graph Resolution — Kahn's Algorithm (Rust)

```
pub fn resolve_order(graph: &DependencyGraph)
    -> Result<ResolveResult, PlannerError>
{
    let mut in_degree: HashMap<&str, usize> = HashMap::new();
    for node in graph.nodes() { in_degree.insert(node, 0); }
    for (_, deps) in graph.edges() {
        for dep in deps { *in_degree.entry(dep).or_insert(0) += 1; }
    }
    let mut queue: VecDeque<&str> = in_degree.iter()
        .filter(|(_, &d)| d == 0).map(|(n, _)| *n).collect();
    let mut order = Vec::new();
    while let Some(node) = queue.pop_front() {
        order.push(node.to_string());
        if let Some(deps) = graph.dependents(node) {
            for dep in deps {
                let d = in_degree.get_mut(dep).unwrap();
                *d -= 1;
                if *d == 0 { queue.push_back(dep); }
            }
        }
    }
    if order.len() != graph.node_count() {
        return Err(PlannerError::CyclicDependency(
            "Cycle detected in dependency graph".into()));
    }
    Ok(ResolveResult { order })
}
```

8.2 Path Traversal Guard (C)

```
static int path_has_traversal(const char *path) {
    const char *p = path;
    while (*p) {
        while (*p == '/') p++;
        if (p[0] == '.' && p[1] == '.' &&
            (p[2] == '/' || p[2] == ''))
            return 1;
        while (*p && *p != '/') p++;
    }
    return 0;
}

/* Usage in execute_step() - OP_COPY handler: */
if (path_has_traversal(src) || path_has_traversal(dst)) {
    log_err("COPY rejected: path traversal in src='%s' dst='%s'",
           src, dst);
    return -1;
}
```

8.3 Supervisor SIGCHLD Handling (C)

```
/* Self-pipe write end (global, write-only in signal handler) */
static int sigchld_pipe_w = -1;

static void sigchld_handler(int sig) {
    (void)sig;
    /* async-signal-safe: write single byte to pipe */
    char b = 1;
    write(sigchld_pipe_w, &b, 1);
}

/* In main() - setup */
int pipefd[2];
pipe2(pipefd, O_NONBLOCK | O_CLOEXEC);
sigchld_pipe_w = pipefd[1];
signal(SIGCHLD, sigchld_handler);

/* In main loop - reap children when pipe is readable */
if (FD_ISSET(pipefd[0], &rfdsets)) {
    char buf[64]; read(pipefd[0], buf, sizeof(buf)); /* drain */
    pid_t pid; int st;
    while ((pid = waitpid(-1, &st, WNOHANG)) > 0)
        sup_handle_dead(pid, st);
}
```

8.4 Service State Machine — sup_handle_dead() (C)

```
void sup_handle_dead(pid_t pid, int status) {
    struct service *svc = find_service_by_pid(pid);
    if (!svc) { fprintf(stderr, "orphan pid=%d", pid); return; }
    int ec = WIFEXITED(status) ? WEXITSTATUS(status)
        : WIFSIGNALED(status) ? -WTERMSIG(status) : 0;
    svc->exit_code = ec;
    svc->pid = -1;

    if (svc->type == SVC_TYPE_ONESHOT) {
        svc->state = (ec == 0) ? SVC_DONE : SVC_FAILED; return;
    }
    if (svc->state == SVC_STOPPED) return; /* explicit stop */

    int should = 0;
    switch (svc->restart) {
        case SVC_RESTART_ALWAYS: should = 1; break;
        case SVC_RESTART_ON_FAILURE: should = (ec != 0); break;
        default: should = 0; break;
    }
    if (should) {
        svc->state = SVC_RESTARTING;
        svc->restart_at = time(NULL) + svc->restart_delay;
    } else {
        svc->state = (ec == 0) ? SVC_STOPPED : SVC_FAILED;
    }
}
```

8.5 Plan Validation (C)

```
int plan_validate(const void *base, size_t sz) {
    if (sz < sizeof(struct plan_header)) {
        log_err("Plan too small: %zu bytes", sz); return -1; }
    const struct plan_header *h = base;
    if (memcmp(h->magic, PLAN_MAGIC, 8) != 0) {
        log_err("Bad magic"); return -1; }
    if (h->version != PLAN_VERSION) {
        log_err("Version %u != %u", h->version, PLAN_VERSION);
        return -1; }
    size_t steps_end = sizeof(*h) +
        (size_t)h->step_count * sizeof(struct
step_record);
    if (steps_end > sz) { log_err("step_count overflows"); return -1; }
    if (h->strtab_offset + h->strtab_len > sz) {
        log_err("String table out of bounds"); return -1; }
    return 0;
}
```

8.6 Technology Stack

Component	Language	Key Dependencies
cogman-planner	Rust (stable)	serde, toml, clap 4.4
cogman-executor	C11 (GCC)	POSIX stdlib only
cogman-supervisor	C11 (GCC)	POSIX stdlib only
cogman-ctl	C11 (GCC)	POSIX stdlib only
AI Advisor	Python + llama.cpp	Qwen2.5-3B (4-bit GGUF)
Rootfs base	BusyBox 1.36.1	statically linked
Test platform	QEMU 8.x	x86_64 emulation

CHAPTER 9

SYSTEM TESTING

9.1 Planner Unit Tests — Schema Validation

TC ID	Input Condition	Expected Behaviour	Result
SV-01	Valid complete package.toml	Exit 0, plan written successfully	PASS
SV-02	Missing [identity] section	Exit 1, serde deserialization error	PASS
SV-03	identity.name = empty string	Exit 1, name must not be empty	PASS
SV-04	identity.version = empty string	Exit 1, version error	PASS
SV-05	build.steps = [] (empty list)	Exit 1, steps must not be empty	PASS
SV-06	policy.filesystem.write = ['./etc']	Exit 1, non-absolute path rejected	PASS
SV-07	Circular dependency $A \rightarrow B \rightarrow A$	Exit 1, cycle detected with path	PASS
SV-08	Dependency not found in packages/	Exit 1, missing dependency error	PASS
SV-09	TOML syntax error (unclosed quote)	Exit 1, TOML parse error with line number	PASS
SV-10	identity.depends.build contains empty string	Exit 1, empty dependency name	PASS

9.2 Executor Unit Tests — Step Operations

TC ID	Op	Test Condition	Expected Result	Result
EX-01	OP_EXEC	echo 'hello' command	stdout contains 'hello', exit 0	PASS
EX-02	OP_EXEC	exit 1, FAIL_ABORT flag	Executor exits with code 1	PASS
EX-03	OP_EXEC			PASS

		exit 1, FAIL_WARN flag	Continues execution, logs warning	
EX-04	OP_MKDIR	Create /tmp/test/a/b/ c (nested)	Directory exists after execution	PASS
EX-05	OP_MKDIR	Already exists (idempotent test)	Exit 0, no error	PASS
EX-06	OP_COPY	Copy file to valid destination	Destination file matches source (byte-exact)	PASS
EX-07	OP_COPY	Destination path contains '..'	Rejected at path_has_traversal, exit 2	PASS
EX-08	OP_COPY	Recursive directory tree copy	All files copied with correct permissions	PASS
EX-09	OP_VERIFY	Existing file path	Exit 0 (file found)	PASS
EX-10	OP_VERIFY	Non-existent path, FAIL_ABORT	Exit 1 (verification failed)	PASS
EX-11	OP_CLEANUP	Remove existing temp directory	Directory gone after execution	PASS
EX-12	Header	Wrong magic bytes in plan file	Exit 1 with "Bad magic" error	PASS
EX-13	Header	Wrong version number	Exit 1 with "Version mismatch" error	PASS

9.3 Supervisor Test Cases — Service Lifecycle

TC ID	Scenario	Setup	Expected Outcome	Result
SL-01	Oneshot completes successfully	hello.service, type=oneshot, cmd=echo ok	State transitions to SVC_DONE	PASS
SL-02	Oneshot fails, restart=never	Command exits code 1	State transitions to SVC_FAILED	PASS
SL-03	Long-running service tracked by PID	sleep 60 command	svc->pid positive, state=RUNNING	PASS
SL-04	restart=always after SIGKILL	Kill service process with SIGKILL	Supervisor restarts within delay+1s	PASS

SL-05	restart=on-failure, clean exit 0	Service exits with code 0	No restart triggered	PASS
SL-06	restart=on-failure, exit code 1	Service exits with code 1	Restart triggered after delay	PASS
SL-07	Dependency gate blocks start	B depends on A; A not yet started	B remains in STOPPED state	PASS
SL-08	Dependency gate opens on A completion	A completes (SVC_DONE)	B starts automatically	PASS
SL-09	Dependency chain A → B → C	Three services with sequential depends	Start order: A, then B, then C	PASS
SL-10	Explicit stop disables restart	cogman-ctl stop service	Service does not restart after termination	PASS
SL-11	Orphan process reaping	Service forks a grandchild before exiting	Grandchild reaped when orphaned (PID 1 property)	PASS
SL-12	SIGTERM initiates clean shutdown	kill -TERM <PID_1>	All services stopped; supervisor exits cleanly	PASS

9.4 End-to-End Boot Tests (QEMU)

TC ID	Boot Condition	Expected Console Output	Result
E2E-01	Normal boot, all services enabled	[SERVICE:hello] cogman-supervisor is alive	PASS
E2E-02	Normal boot	[SERVICE:heartbeat] tick 0	PASS
E2E-03	Normal boot	[SERVICE:ctl-probe] control socket OK	PASS
E2E-04	Normal boot	[SERVICE:exec-probe] plan execution OK	PASS
E2E-05	Kill heartbeat with SIGKILL	Supervisor restarts heartbeat within 3 s	PASS
E2E-06	cogman-ctl list command	All 4 services listed with correct states	PASS
E2E-07	cogman-ctl stop heartbeat	heartbeat state: stopped (no restart)	PASS
E2E-08		All services start; no OOM kill events	PASS

	Boot with 64 MB RAM constraint		
--	--------------------------------	--	--

9.5 User Acceptance Testing

TC ID	User Story	Acceptance Criterion	Result
UAT-01	As a package author, I want clear errors for malformed TOML	Error message identifies missing field and line number	PASS
UAT-02	As a build engineer, I want circular deps detected early	Planner reports cycle path before any execution	PASS
UAT-03	As an operator, I want to restart a crashed service	cogman-ctl restart <name> works within 1 s	PASS
UAT-04	As an operator, I want path traversal attempts blocked	OP_COPY with '..' in path rejected with clear error	PASS
UAT-05	As a developer, I want fast re-planning on unchanged packages	Cache hit produces plan in <1 ms	PASS

CHAPTER 10

PERFORMANCE ANALYSIS

10.1 Plan Resolution Time

The $56\times$ improvement in plan resolution time (from ~ 450 ms to ~ 8 ms) arises from three compounding factors. First, the Rust planner avoids Python's interpreter startup overhead of approximately 50 ms per invocation — a fixed cost independent of the size of the package metadata being parsed. Second, TOML deserialization using `serde`'s `derive` macro produces compiled deserialization code that directly constructs Rust structs from the TOML byte stream without intermediate object allocation; Python's pure-Python `toml` library constructs a Python dict hierarchy through multiple layers of interpretation, allocation, and reference counting. Third, the dependency graph algorithms use Rust's `HashMap` and `Vec` with $O(1)$ average-case lookup and cache-friendly memory layout, compared to Python's dict-based implementation with pointer-following indirection and per-object reference count updates.

The content-addressed plan cache provides an additional performance benefit on repeated build invocations. The FNV-1a hash is computed over the package name, version string, and the content of the TOML file — approximately 0.3 ms for a typical 2–5 KB package metadata file. On a cache hit, the entire plan resolution is short-circuited to this hash computation plus a file existence check, reducing the cost from 8 ms to 0.3 ms — a $27\times$ reduction for the common case in CI/CD environments where most packages are unchanged between build runs.

10.2 Memory Usage

The $21\times$ reduction in peak memory (from ~ 85 MB to ~ 4 MB) is primarily attributable to Python's per-object allocation overhead. In CPython 3.11, every Python object carries a `ob_refcnt` (8 bytes), an `ob_type` pointer (8 bytes), and variable additional fields. A Python `str` object for a 20-character string requires

approximately 73 bytes — $3.65\times$ the string content. A Python list object carrying 100 package metadata dicts requires substantially more memory than the equivalent Rust Vec because each Python dict entry carries a PyObject* pointer, a hash value, and reference count overhead in addition to the key and value data. Rust's serde-derived struct layout allocates exactly the memory required by the struct fields with no overhead beyond standard alignment padding.

10.3 Per-Step Execution Overhead

The $50\times$ reduction in per-step execution overhead (from ~ 45 ms to ~ 0.9 ms) is dominated by the elimination of Python's subprocess.run() overhead. Python's subprocess.run() constructs a subprocess.CompletedProcess object, performs multiple Python type checks and attribute lookups, calls os.fork() through the Python C API, and awaits the child through a polling loop. The C executor's execute_step() function calls fork() and execve() directly via the C standard library wrappers, taking approximately 0.5–1.0 ms — essentially the minimum achievable process creation overhead on a modern x86_64 processor, limited by the kernel's process table allocation and scheduler dispatch costs.

10.4 Performance Summary

Metric	Legacy Python	Rogue Linux (Rust/C)	Improvement
Plan resolution (cold)	~ 450 ms	~ 8 ms	$56\times$
Plan resolution (cache hit)	~ 450 ms (no cache)	~ 0.3 ms	$1500\times$
Peak planner memory	~ 85 MB	~ 4 MB	$21\times$
Per-step execution overhead	~ 45 ms	~ 0.9 ms	$50\times$
Minimal rootfs size	—	~ 6.3 MB	—
QEMU boot to first service	—	< 500 ms	—
plan_validate() time (5 MB plan)	—	< 1 ms	—

10.4.1 Build System Comparison

To provide broader context for the Rogue Linux performance figures, a comparison against Buildroot and the Python reference implementation is informative. A Buildroot build of a similarly minimal rootfs (BusyBox + musl + Linux kernel, no external packages) takes approximately 25–40 minutes on a 4-core build host and requires approximately 8 GB of disk space for the build tree. The Rogue Linux build pipeline for the same package set takes approximately 3–8 minutes (dominated by the Linux kernel compilation) with less than 1 GB of disk space for the Cogman build tree, because the planner and executor do not maintain a separate per-package stamp directory tree.

The key architectural difference that enables this reduction is that Rogue Linux does not implement its own download manager, patch system, or compiler toolchain wrapper — these are provided by the package author's build steps and the host system's existing toolchain. This design choice sacrifices some of Buildroot's generality (e.g., built-in cross-compilation support with configured toolchain paths) in favor of a much simpler and faster build orchestration layer.

10.5 Rootfs Size Analysis

The 6.3 MB minimal rootfs is composed of five categories of content. The BusyBox binary accounts for approximately 1.2 MB (stripped, x86_64). The Cogman supervisor binary accounts for approximately 180 KB (stripped). The dynamic libraries (libc.so.6, ld-linux-x86-64.so.2, libgcc_s.so.1) account for approximately 3.8 MB. The /etc/cogman/ service definition files, boot scripts, and plan files account for approximately 20 KB. The /dev, /proc, /sys, and /tmp directory scaffolding accounts for negligible space. A musl libc-based build eliminates the glibc dynamic library dependency entirely, reducing the rootfs to approximately 4.1 MB — competitive with Alpine Linux (~5 MB).

CHAPTER 11

CONCLUSION AND FUTURE WORK

11.1 Conclusion

This project has successfully designed, implemented, and validated Rogue Linux — a deterministic, metadata-driven infrastructure for constructing minimal Linux-based operating system images, with Cogman as the unified toolchain spanning both build and runtime phases. The project delivers on all stated objectives: a schema-validated TOML package metadata format, a Rust-based cogman-planner with DAG resolution and CGM2PLAN binary emission, a C11-based cogman-executor with typed step operations and path traversal protection, a POSIX-correct PID-1 supervisor with dependency-aware service management and SIGCHLD self-pipe child reaping, a text-protocol Unix domain socket control interface, and a bootable minimal rootfs of approximately 6.3 MB verified under QEMU.

The performance evaluation demonstrates all three headline improvements over the legacy Python baseline: 56× faster plan resolution (8 ms vs. 450 ms), 21× lower peak memory (4 MB vs. 85 MB), and 50× lower per-step execution overhead (0.9 ms vs. 45 ms). The content-addressed plan cache further reduces planning time to 0.3 ms on unchanged packages, a critical optimization for CI/CD environments. All 40 unit, integration, supervisor lifecycle, and end-to-end test cases pass on the QEMU test platform.

The system demonstrates that high-performance, memory-safe, and reproducible system software can be built using modern Rust and C11 without external framework dependencies, and that the build-time and runtime halves of an embedded Linux system can be unified under a single coherent metadata model that improves auditability, reproducibility, and security policy enforcement simultaneously.

11.1.1 Assessment Against Objectives

All seven stated project objectives have been met. The TOML package definition format with schema validation was designed and implemented with compile-time type checking via `serde's derive` macro, providing clear deserialization errors for all required fields. The `cogman-planner` achieves the stated 50× performance target with a 56× improvement in plan resolution time. The `cogman-executor` correctly handles all five typed step operations with path traversal protection on `OP_COPY`, verified by 13 unit tests. The `cogman-supervisor` correctly implements PID-1 semantics with `SIGCHLD` self-pipe reaping, verified by 12 service lifecycle tests and 8 end-to-end boot tests under QEMU. The `cogman-ctl` control interface functions correctly for all five command types over the Unix domain socket, verified by end-to-end tests E2E-06 through E2E-08. The minimal rootfs of 6.3 MB is below the 10 MB target and boots successfully under QEMU with 64 MB RAM. The 56× plan resolution improvement exceeds the stated 50× target.

The most technically challenging aspect of the implementation was the `SIGCHLD` self-pipe pattern in `cogman-supervisor`. The initial implementation used a direct `waitpid()` call in the `SIGCHLD` handler, which passed functional tests but was theoretically unsafe. The production implementation was refactored to use the self-pipe pattern after identifying the `async-signal-safety` requirement, and the change required updates to the main `select()` loop, the signal handler, and the child reaping logic — a non-trivial refactor that was validated by the full supervisor test suite before merging.

11.2 Summary of Technical Contributions

The four primary technical contributions are: (1) the **CGM2PLAN binary plan format**, which provides a portable, zero-parsing-overhead interface between the Rust planner and C executor; (2) the **SIGCHLD self-pipe pattern** for safe PID-1 child reaping that avoids signal-handler/main-loop race conditions; (3) the

content-addressed plan cache that eliminates redundant plan re-computation for unchanged packages; and (4) the **path traversal guard** that provides a structural safety property for all OP_COPY operations regardless of the plan content.

11.2.1 Limitations

The current system has four documented limitations. First, the argument-level policy enforcement in cogman-executor checks path traversal but does not validate that OP_EXEC commands are restricted to a declared command allowlist. A malicious package.toml could include arbitrary shell commands in the build steps, which would execute during build time with the build host's permissions. Production deployment requires either a command allowlist in the policy schema or a dedicated build sandbox (e.g., bubblewrap or a network-isolated container) for each package build.

Second, the cogman-supervisor does not yet implement cgroup-based resource limits per service. A runaway service can consume all available CPU or memory, affecting other services. Integration with Linux cgroups v2 (available since kernel 4.5) would provide per-service resource limits configurable through the service definition file.

Third, the IPC messenger protocol does not implement authentication or access control: any process that can connect to the Unix domain socket at /tmp/cogman.sock can issue any control command. In a multi-user environment, the socket file should be protected by DAC (discretionary access control, e.g., chmod 0600 with root ownership) and the supervisor should validate the connecting process's UID via SO_PEERCRED before accepting commands from non-root clients.

Fourth, the AI advisor component (Qwen2.5-3B) is implemented as a separate process that queries the LLM on request. The advisor has no write access to any Cogman state and cannot issue commands to the supervisor, but the quality

of its responses depends on the Qwen2.5-3B model's training data for Cogman-specific content. Fine-tuning on a Cogman-specific dataset is required to achieve production-quality advisory accuracy.

11.3 Future Work

Landlock filesystem isolation: Restrict each service to its declared filesystem policy paths using the Linux Landlock LSM (available since Linux 5.13), providing per-service mandatory access control without the complexity of SELinux or AppArmor policy authoring.

seccomp-BPF system call filtering: Generate a per-service seccomp filter from the service definition's declared syscall set, reducing the kernel attack surface by restricting each service to the minimum set of system calls required for its operation.

Linux namespace isolation: Provide network and PID namespace isolation for services that declare isolation requirements in their service definition files, enabling lightweight container-like isolation without a full container runtime.

ARM64 and RISC-V cross-compilation: Extend the build system to support cross-compilation to ARM64 (Cortex-A series) and RISC-V (RV64GC) targets, enabling Rogue Linux rootfs images for the embedded targets that constitute the primary production deployment environment for minimal Linux images.

Qwen2.5 QLoRA fine-tuning: Fine-tune the Qwen2.5-3B advisor model on a curated dataset of Cogman error messages, package.toml examples, and service file configurations using QLoRA, improving the accuracy of AI-assisted troubleshooting for common build and configuration errors.

Incremental build support: Extend the content-addressed cache to track build outputs as well as plan content, enabling skip-if-unchanged optimization at

the individual build step level for packages whose source archives have not changed between build invocations.

APPENDIX — COMPLETE SOURCE CODE LISTINGS

A.1 Package Metadata Schema — Rust Struct Definitions (schema.rs)

```

#[derive(Debug, Deserialize, Serialize, Clone)]
pub struct PackageMetadata {
    pub identity: Identity,
    pub build:    Builder,
    pub installer: Installer,
    #[serde(default)]
    pub policy:   Policy,
}

#[derive(Debug, Deserialize, Serialize, Clone)]
pub struct Identity {
    pub name:      String,
    pub version:   String,
    pub category:  String,
    pub summary:   String,
    pub source:    Source,
    #[serde(default)]
    pub depends:   Depends,
}

#[derive(Debug, Deserialize, Serialize, Clone)]
pub struct Source {
    pub kind: SourceKind,
    pub file: String,
}

#[derive(Debug, Deserialize, Serialize, Clone, Copy, PartialEq)]
#[serde(rename_all = "lowercase")]
pub enum SourceKind { Tarball, Git }

#[derive(Debug, Deserialize, Serialize, Default, Clone)]
pub struct Depends {
    #[serde(default)] pub build:    Vec<String>,
    #[serde(default)] pub runtime: Vec<String>,
}

#[derive(Debug, Deserialize, Serialize, Clone)]
pub struct Builder {
    pub system: BuildSystem,
    pub steps:  Vec<String>,
}

#[derive(Debug, Deserialize, Serialize, Clone)]
pub struct Installer {
    pub steps: Vec<String>,
    pub verify: Option<Verify>,
}

#[derive(Debug, Deserialize, Serialize, Default, Clone)]
pub struct Policy {
    pub filesystem: Filesystem,
    pub network:    Network,
}

```

```
#[derive(Debug, Deserialize, Serialize, Default, Clone)]
pub struct Filesystem {
    #[serde(default)] pub write: Vec<String>,
}

#[derive(Debug, Deserialize, Serialize, Default, Clone)]
pub struct Network {
    #[serde(default)] pub outbound: bool,
}
```

A.2 Example package.toml — BusyBox 1.36.1

```
[identity]
name      = "busybox"
version   = "1.36.1"
category  = "base"
summary   = "Minimal Unix utility suite for embedded Linux systems"

[identity.source]
kind = "tarball"
file = "busybox-1.36.1.tar.bz2"

[identity.depends]
build   = ["musl"]
runtime = []

[build]
system = "make"
steps  = [
    "make defconfig",
    "echo 'CONFIG_STATIC=y' >> .config",
    "make -j$(nproc) CONFIG_PREFIX=$PKGROOT",
]

[installer]
steps = [
    "mkdir -p $PKGROOT/bin",
    "cp busybox $PKGROOT/bin/busybox",
    "ln -s busybox $PKGROOT/bin/sh",
    "ln -s busybox $PKGROOT/bin/ls",
    "ln -s busybox $PKGROOT/bin/cat",
    "ln -s busybox $PKGROOT/bin/echo",
]

[installer.verify]
expected_files = ["bin/busybox", "bin/sh"]

[policy.filesystem]
write = ["/mnt/rogue/pkgroot/base/busybox"]

[policy.network]
outbound = false
```

A.3 Example Service Definition — heartbeat.service

```
[service]
name      = "heartbeat"
command   = "/usr/bin/cogman-heartbeat"
type      = "simple"
restart   = "always"
restart_delay = 3
depends_on = []

[env]
TICK_INTERVAL = "5"
LOG_LEVEL     = "info"

[meta]
description = "Periodic liveness signal for system health monitoring"
enabled     = true
```

A.4 Minimal Linux Distribution Manifest (minimal-linux.toml)

```
[distro]
name      = "rogue-linux-minimal"
version   = "0.1.0"
arch      = "x86_64"
summary   = "Minimal Rogue Linux – cogman verification target"

[[packages]]
name = "musl"; version = "1.2.5"; category = "base"
toml = "musl/musl.toml"

[[packages]]
name = "busybox"; version = "1.36.1"; category = "base"
toml = "busybox/busybox.toml"

[[packages]]
name = "linux-kernel"; version = "6.6.30"; category = "base"
toml = "linux-kernel/linux-kernel.toml"

[[packages]]
name = "cogman-exec"; version = "1.0.0"; category = "cogman"
toml = "cogman-exec/cogman-exec.toml"

[[packages]]
name = "cogman-planner"; version = "1.0.0"; category = "cogman"
toml = "cogman-planner/cogman-planner.toml"

[[packages]]
name = "cogman-supervisor"; version = "1.0.0"; category = "cogman"
toml = "cogman-supervisor/cogman-supervisor.toml"

[rootfs]
makedirs = ["/bin", "/sbin", "/usr/bin", "/lib",
            "/etc/cogman/services", "/etc/cogman/plans",
            "/run", "/tmp", "/proc", "/sys", "/dev", "/root"]
init     = "/sbin/init"

[image]
kernel  = "/boot/vmlinuz"
cmdline = "console=ttyS0 root=/dev/sda rw init=/sbin/init quiet"
```

A.5 cogman-ctl Command Reference

USAGE: cogman-ctl [--socket PATH] COMMAND

COMMANDS:

list		List all services with PID, state, restart count
start	<name>	Start a STOPPED service
stop	<name>	Send SIGTERM to service; mark STOPPED (no restart)
restart	<name>	Stop then start a service
status	<name>	Detailed status: state, PID, exit code, uptime
reload		Re-read service definition files (SIGHUP to supervisor)
shutdown		Send SIGTERM to supervisor (graceful shutdown)

PROTOCOL (Unix domain socket, text-based):

```
Client sends: "command arg1 arg2"
"
Server sends: "<response lines>"
OK
" or "ERR <message>"
"
```

EXAMPLES:

```
cogman-ctl list
cogman-ctl stop heartbeat
cogman-ctl start heartbeat
cogman-ctl status heartbeat
cogman-ctl restart exec-probe
```

REFERENCES

1. Kerrisk, M. (2010). *The Linux Programming Interface*. No Starch Press.
2. Klabnik, S., and Nichols, C. (2019). *The Rust Programming Language*. No Starch Press.
3. Ritchie, D. M., and Thompson, K. (1974). The UNIX Time-Sharing System. *Communications of the ACM*, 17(7), 365–375.
4. Stevens, W. R., and Rago, S. A. (2013). *Advanced Programming in the UNIX Environment*, 3rd ed. Addison-Wesley.
5. Preston-Werner, T. (2023). *TOML: Tom's Obvious Minimal Language v1.0.0 Specification*. toml.io.
6. Bellard, F. (2005). QEMU, a Fast and Portable Dynamic Translator. *USENIX Annual Technical Conference*.
7. Wheeler, D. A. (2003). *Secure Programming HOWTO*. dwheeler.com/secure-programs.
8. Drepper, U. (2013). *ELF: Executable and Linking Format*. Tool Interface Standard Committee.
9. Kahn, A. B. (1962). Topological sorting of large networks. *Communications of the ACM*, 5(11), 558–562.
10. Bernstein, D. J. (1991). FNV Hash Function. isthe.com/chongo/tech/comp/fnv.
11. BusyBox Project. (2023). *BusyBox — The Swiss Army Knife of Embedded Linux*. busybox.net.
12. Alibaba Cloud. (2024). *Qwen2.5 Technical Report*. arxiv.org/abs/2412.15115.
13. Hu, E. J., et al. (2022). LoRA: Low-Rank Adaptation of Large Language Models. *ICLR 2022*.
14. Soltesz, S., et al. (2007). Container-based Operating System Virtualization. *EuroSys 2007*.
15. Dolev, D., and Yao, A. C. (1983). On the security of public key protocols. *IEEE Trans. Information Theory*, 29(2).